

Dan Boneh 密码学第一季：第十一课 基于陷门置换的公钥加密

本笔记按照课程讲义的主线整理：公钥加密安全定义 → 陷门函数与混合加密 → RSA 陷门置换 → 教科书 RSA 攻击 → PKCS#1 v1.5 与 Bleichenbacher 攻击 → OAEP → RSA 假设、参数与实现攻击。

1. 本课的核心问题

上一课介绍了模运算、模根、离散对数和因数分解。本课利用这些数学工具构造公钥加密，并回答：

一个任何人都能正向计算的公开函数，怎样让只有私钥持有者能够高效求逆？

这种对象称为陷门函数或陷门置换。但本课最重要的结论不是“RSA 可以加密”，而是：

陷门置换本身不是安全公钥加密。直接计算 $c = m^e \bmod N$ 的教科书 RSA 是确定的、可篡改的，也不具有语义安全。

要构造安全系统，还需要：

- 随机化；
- 安全填充或编码；
- 密钥派生；
- 对称认证加密；
- 抵抗选择密文攻击的解密接口；
- 不泄露错误类型和运行时间的实现。

学完本课，应当能够回答：

1. 公钥加密的 IND-CPA 与 IND-CCA 安全分别是什么？
2. 陷门函数、单向函数和陷门置换有什么区别？
3. 如何通过陷门函数构造 KEM-DEM 风格的混合加密？
4. RSA 为什么是一个置换，私钥为什么能求逆？
5. RSA 单向假设与因数分解有什么关系？
6. 教科书 RSA 为什么不具有语义安全？
7. RSA 的乘法结构如何造成攻击？
8. PKCS#1 v1.5 的填充预言机如何解密密文？
9. OAEP 的随机化和完整性检查分别起什么作用？
10. 小私钥、侧信道、故障和弱随机数为什么能破坏 RSA？

2. 公钥加密回顾

2.1 三个算法

公钥加密系统由三个算法组成：

$$\Pi = (G, E, D).$$

密钥生成

$$(pk, sk) \leftarrow G().$$

G 是随机算法：

- pk 是公开密钥；
- sk 是秘密密钥。

加密

$$c \leftarrow E(pk, m).$$

安全公钥加密的 E 通常必须是随机算法。

解密

$$m \text{ 或 } \perp \leftarrow D(sk, c).$$

$\perp$ 表示密文格式或认证无效，接收方拒绝。

2.2 正确性

对合法密钥对和消息，要求：

$$D(sk, E(pk, m)) = m.$$

严格定义中，等式应对加密算法所有合法随机选择成立，或者以压倒性概率成立。

2.3 典型用途

会话建立

发送方选择随机秘密 x ，使用接收方公钥加密：

$$c = E(pk, x).$$

接收方解密得到 x ，双方再通过 KDF 派生会话密钥。

非交互通信

接收方提前公开公钥，发送方无需与接收方同时在线即可加密邮件或文件。

前提是发送方能够确认该公钥确实属于目标接收方，这属于公钥管理和身份认证问题。

3. 公钥加密的安全定义

3.1 IND-CPA：抵抗窃听

公钥语义安全也称 IND-CPA 安全。实验如下：

1. 挑战者生成：

$$(pk, sk) \leftarrow G();$$

2. 把 pk 交给攻击者;
3. 攻击者提交两条等长消息 m_0, m_1 ;
4. 挑战者随机选择 $b \in \{0, 1\}$, 返回:

$$c^* \leftarrow E(pk, m_b);$$

5. 攻击者猜测 b 。

优势为:

$$\text{Adv}_{\text{IND-CPA}}[A, \Pi] = |\Pr[A \text{ 在 } \text{EXP}(0) \text{ 输出 } 1] - \Pr[A \text{ 在 } \text{EXP}(1) \text{ 输出 } 1]|.$$

若所有高效攻击者的优势都可忽略, 则方案是 IND-CPA 安全的。

3.2 为什么公钥加密必须随机化

攻击者知道 pk , 可以自己运行加密算法。若加密是确定性的, 它收到挑战密文后可直接计算:

$$E(pk, m_0) \text{ 和 } E(pk, m_1),$$

然后与 c^* 比较, 立即得到 b 。

因此:

确定性公钥加密不可能满足普通 IND-CPA 安全。

3.3 为什么一次安全可以推出多次安全

在公钥环境中, 攻击者本来就能自行加密任意消息。通过混合实验, 可以一次替换一个挑战密文; 若攻击者能够区分很多次加密, 就能用它区分其中某一次。

因此, 适当定义下的公钥一次语义安全可以推出多次 CPA 安全。

3.4 IND-CCA: 抵抗主动选择密文攻击

在 IND-CCA 实验中, 攻击者还可以访问解密接口:

1. 挑战前, 可以提交任意密文并得到解密结果;
2. 收到挑战密文 c^* 后, 仍可以提交任意 $c \neq c^*$;
3. 唯一禁止的是把 c^* 原样交给解密接口。

攻击目标仍是判断 c^* 加密的是 m_0 还是 m_1 。

若其优势可忽略, 则方案是 IND-CCA 安全的。

3.5 为什么公钥环境不能要求“攻击者不能制造新密文”

在对称认证加密中, 攻击者不知道密钥, 密文完整性可以要求它不能产生新的有效密文。

但公钥加密的 pk 对所有人公开, 攻击者本来就能运行:

$$E(pk, m)$$

制造任意合法新密文。因此，公钥保密性通常直接通过 IND-CCA 定义主动攻击安全，而不是要求普通意义上的密文不可伪造。

4. 陷门函数

4.1 定义

陷门函数族由三个高效算法组成：

$$(G, F, F^{-1}).$$

参数和陷门生成

$$(pk, sk) \leftarrow G().$$

pk 描述公开函数， sk 是求逆所需的陷门。

正向计算

$$y = F(pk, x), \quad F(pk, \cdot) : X \rightarrow Y.$$

任何知道 pk 的人都能高效计算。

使用陷门求逆

$$x = F^{-1}(sk, y).$$

正确性要求：

$$F^{-1}(sk, F(pk, x)) = x.$$

4.2 单向性

安全实验为：

1. 生成 (pk, sk) ;
2. 随机选择 $x \leftarrow X$;
3. 计算 $y = F(pk, x)$;
4. 只把 pk, y 交给攻击者;
5. 攻击者尝试输出原来的 x 。

单向优势为：

$$\text{Adv}_{OW}[A, F] = \Pr[A(pk, F(pk, x)) = x].$$

若对所有高效攻击者该概率都可忽略，则称陷门函数是单向的。

4.3 陷门置换

若 $X = Y$ ，并且对每个公钥 pk ， $F(pk, \cdot)$ 是双射，则称为 Trapdoor Permutation，简称 TDP。

对象	正向是否高效	无陷门是否难逆	是否必须一一映射
单向函数	是	是	否
陷门函数	是	是	否，有陷门可逆
陷门置换	是	是	是，有陷门可唯一求逆

5. 从陷门函数构造公钥加密

5.1 所需组件

设有：

- 1. 安全陷门函数 (G, F, F^{-1}) ;
- 2. 对称认证加密 (E_s, D_s) ;
- 3. 哈希或密钥派生函数：

$$H : X \rightarrow K.$$

讲义在随机预言机模型中分析 H 。

5.2 加密

对消息 m ：

- 1. 随机选择：

$$x \leftarrow X;$$

- 2. 计算陷门函数输出：

$$y = F(pk, x);$$

- 3. 从 x 派生对称密钥：

$$k = H(x);$$

- 4. 使用认证加密保护消息：

$$c = E_s(k, m);$$

- 5. 输出：

$$(y, c).$$

5.3 解密

收到 (y, c) ：

- 1. 使用陷门恢复：

$$x = F^{-1}(sk, y);$$

- 2. 派生相同密钥：

$$k = H(x);$$

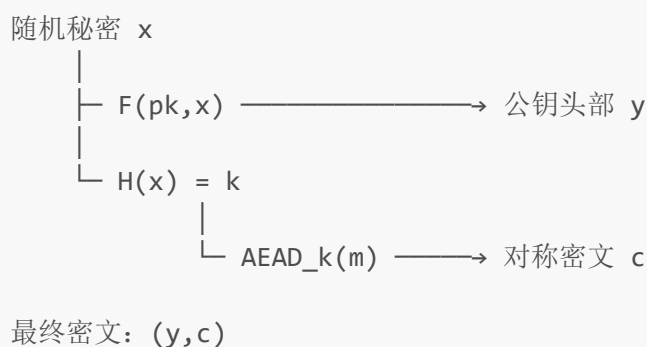
3. 认证解密:

$$m \text{ 或 } \perp = D_s(k, c).$$

5.4 KEM-DEM 视角

这个构造可以分成:

- **KEM**: 利用公钥封装随机秘密 x 或密钥 k , 头部是 y ;
- **DEM**: 利用派生出的对称密钥加密实际消息, 主体是 c 。



公钥运算只处理短随机秘密, 大消息由高效的对称认证加密处理。这就是混合加密的基本思想。

5.5 安全直觉

- 陷门函数单向性使攻击者难以从 y 恢复 x ;
- 随机预言机 $H(x)$ 在攻击者未找到 x 时看起来像随机密钥;
- 对称认证加密隐藏 m , 并拒绝被修改的主体密文;
- 随机选择 x 让同一消息每次加密得到不同结果。

讲义给出的定理是: 若陷门函数安全、对称方案提供认证加密, 并把 H 建模为随机预言机, 则该构造在相应随机预言机模型中实现 CCA 安全。

5.6 随机预言机模型

理想随机预言机是一个所有人都能查询、但对新输入返回独立随机输出并保持一致性的理想函数。

实际系统使用 SHA-256、SHA-3 等具体哈希或标准 KDF 来实例化, 但“具体哈希等于真正随机预言机”不是一个数学事实。安全结论应理解为随机预言机模型下的证明。

6. 为什么不能直接用陷门函数加密

错误构造为:

$$c = F(pk, m),$$

$$m = F^{-1}(sk, c).$$

它至少存在两个根本问题。

6.1 确定性

同一消息总产生同一密文：

$$m = m' \implies F(pk, m) = F(pk, m').$$

攻击者知道 pk ，可以自行计算候选消息的函数值并比较，因此不可能满足 IND-CPA。

6.2 代数结构和可篡改性

具体陷门置换可能保留乘法等代数关系。攻击者即使不能完全求逆，也可能从密文判断明文关系，或把一个密文转换成另一个相关明文的密文。

所以：

单向性只说明难以完全求逆，不等于语义安全。

7. RSA 陷门置换

7.1 密钥生成

1. 随机选择两个大素数 p, q ;
2. 计算：

$$N = pq;$$

3. 计算：

$$\varphi(N) = (p-1)(q-1);$$

4. 选择 e 满足：

$$\gcd(e, \varphi(N)) = 1;$$

5. 计算：

$$d = e^{-1} \pmod{\varphi(N)};$$

即：

$$ed \equiv 1 \pmod{\varphi(N)}.$$

输出：

$$pk = (N, e), \quad sk = (N, d).$$

实际私钥通常还保存 p, q 和 CRT 加速参数。

7.2 正向置换

RSA 正向函数定义在 $\mathbb{Z}_N^*$ 上：

$$RSA_{N,e}(x) = x^e \pmod{N}.$$

由于 e 与 $\varphi(N)$ 互素，指数映射在 $\mathbb{Z}_N^*$ 上是一一映射。

7.3 使用陷门求逆

给定：

$$y = x^e \pmod{N},$$

私钥持有者计算：

$$y^d = x^{ed} \pmod{N}.$$

因为存在整数 k ：

$$ed = 1 + k\varphi(N),$$

所以对 $x \in \mathbb{Z}_N^*$ ：

$$\begin{aligned} x^{ed} &= x^{1+k\varphi(N)} \\ &= x(x^{\varphi(N)})^k \\ &\equiv x \pmod{N}. \end{aligned}$$

因此：

$$(x^e)^d \equiv x \pmod{N}.$$

7.4 陷门是什么

知道 p, q 就能计算 $\varphi(N)$ ，进而从公开指数 e 求出 d 。只公开 N 时，恢复 p, q 被认为困难。

所以 RSA 的陷门可以理解为：

$$(p, q)$$

或由其得到的私钥指数和 CRT 参数。

8. RSA 单向假设

8.1 假设内容

随机生成大素数 p, q ，令 $N = pq$ ，选择合法 e ，再随机选择：

$$x \leftarrow \mathbb{Z}_N^*,$$

并公开：

$$y = x^e \pmod{N}.$$

RSA 假设认为：没有私钥时，高效攻击者从 (N, e, y) 恢复 x 的概率可忽略。

这等价于难以计算随机 y 的模 N 的 e 次根：

$$x = y^{1/e} \pmod{N}.$$

8.2 知道因数分解就能求逆

若攻击者分解了：

$$N = pq,$$

就能计算 $\varphi(N)$ 、求得 d ，再计算 y^d 。

因此：

能够高效分解 $N \implies$ 能够高效求 RSA 逆。

8.3 RSA 求逆是否等价于因数分解

目前并不知道一般 RSA 求逆一定与因数分解等价。也就是说，尚未证明任何能够求随机 RSA e 次根的算法都可以被转化为通用因数分解算法。

讲义强调这是公钥密码学中历史悠久的问题，并提到对某些“代数型归约”存在否定性证据。

因此应区分：

- RSA 假设：RSA 求逆困难；
- 因数分解假设：分解 N 困难。

二者密切相关，但不是已证明完全等价的同一假设。

9. RSA 混合加密

把第 5 节的一般构造实例化为 RSA：

9.1 加密

1. 随机选择：

$$x \leftarrow \mathbb{Z}_N^*;$$

2. 计算 RSA 头部：

$$y = x^e \pmod{N};$$

3. 派生对称密钥：

$$k = H(x);$$

4. 认证加密消息：

$$c = E_s(k, m);$$

5. 输出：

$$(y, c).$$

9.2 解密

1. 使用私钥恢复：

$$x = y^d \pmod{N};$$

2. 派生：

$$k = H(x);$$

3. 返回：

$$D_s(k, c).$$

认证失败则输出 $\perp$ 。

这种“RSA 封装随机秘密 + 对称 AEAD 加密数据”的结构比直接用 RSA 处理长消息更合理，也体现了现代混合加密的设计方法。

10. 教科书 RSA 为什么不安全

10.1 定义

教科书 RSA 直接把消息当成 $\mathbb{Z}_N$ 元素：

$$c = m^e \pmod{N},$$

解密：

$$m = c^d \pmod{N}.$$

10.2 确定性攻击

攻击者知道 (N, e) 。在 IND-CPA 游戏中，它可以计算：

$$c_0 = m_0^e \pmod{N},$$

$$c_1 = m_1^e \pmod{N},$$

然后把挑战密文与 c_0, c_1 比较。

所以教科书 RSA 不是语义安全的。

10.3 小消息空间枚举

若消息只来自很小的集合，例如四位 PIN 或短会话密钥，攻击者可以枚举所有候选 m' ，计算：

$$(m')^e \pmod{N},$$

再与目标密文比较。

RSA 求逆整体可能困难，但小消息空间仍然可以被穷举。

10.4 乘法同态

RSA 满足：

$$RSA(m_1)RSA(m_2) \equiv (m_1 m_2)^e \pmod{N}.$$

如果：

$$c = m^e,$$

攻击者任选可逆 r ，可构造：

$$c' = c \cdot r^e \pmod{N} = (mr)^e \pmod{N}.$$

解密结果就是：

$$m' = mr \pmod{N}.$$

攻击者虽然不知道 m ，却能有控制地改变明文。因此教科书 RSA 具有可篡改性，不可能 CCA 安全。

10.5 讲义中的 meet-in-the-middle 攻击

假设会话密钥 k 只有 64 位，并且：

$$k = k_1 k_2, \quad k_1, k_2 < 2^{34}.$$

攻击者看到：

$$c = k^e = (k_1 k_2)^e \pmod{N}.$$

若 k_1 可逆，则：

$$c(k_1^e)^{-1} \equiv k_2^e \pmod{N}.$$

攻击者可以：

1. 为所有 $k_1 < 2^{34}$ 建表：

$$c(k_1^e)^{-1} \pmod{N};$$

2. 为所有 $k_2 < 2^{34}$ 计算 k_2^e 并查表；
3. 找到匹配时恢复 $k = k_1 k_2$ 。

讲义估计总攻击量约为 2^{40} ，远低于直接搜索 2^{64} 。

这进一步说明：

RSA 陷门置换不是可以直接使用的加密方案。

11. PKCS#1 v1.5 加密填充

11.1 编码结构

标准 RSAES-PKCS1-v1_5 加密编码的高层形式为：

```
00 || 02 || PS || 00 || M
```

其中：

- **00 02** 是块类型标记；
- **PS** 是至少 8 字节的随机非零填充；
- **00** 是分隔符；
- **M** 是消息；
- 整个编码长度等于 RSA 模数的字节长度。

编码后的整数再进行 RSA 运算。

随机填充解决了教科书 RSA 的确定性问题，但安全性还取决于解密时如何处理格式错误。

11.2 填充有效性成为预言机

服务器收到密文 c 后计算 c^d ，再检查明文是否以合法的 PKCS#1 v1.5 结构开始。

若服务器对“填充合法”和“填充非法”产生可区分行为，攻击者便得到：

$\text{Oracle}(c) = \begin{cases} 1, & \text{RSA}^{-1}(c) \text{ 具有合法 PKCS\#1 格式} \\ 0, & \text{否则} \end{cases}$

它只泄露一个比特，但攻击者可以自适应查询许多次。

12. Bleichenbacher 选择密文攻击

12.1 利用 RSA 的乘法结构

攻击者希望解密目标密文：

$$c = x^e \pmod{N}.$$

它选择不同的 $r \in \mathbb{Z}_N^*$ ，构造：

$$c' = r^e c \pmod{N}.$$

服务器解密得到：

$$(c')^d = rx \pmod{N}.$$

然后预言机告诉攻击者 $rx \bmod N$ 是否落入合法 PKCS#1 编码区间。

12.2 如何逐渐缩小明文范围

合法编码具有固定前缀，意味着解密整数处于某个已知区间。每次选择 r 并观察合法/非法结果，都会对未知 x 给出一个模不等式约束。

攻击者不断：

1. 选择新的 r ；
2. 查询 $r^e c \bmod N$ ；

3. 根据预言机结果更新 x 的可能区间；
4. 直到区间只剩一个整数。

最终无需私钥即可恢复目标明文。

12.3 Baby Bleichenbacher 直觉

讲义使用一个不是真实 RSA 模数的简化例子 $N = 2^n$ 。如果预言机只告诉解密结果最高位是否为 1，那么查询：

$$c, \quad 2^e c, \quad 4^e c, \quad 8^e c, \dots$$

相当于依次观察：

$$x, \quad 2x \bmod N, \quad 4x \bmod N, \dots$$

的最高位，从而逐位恢复 x 。

真实 Bleichenbacher 攻击的区间运算更复杂，但核心相同：**密文可乘性 + 格式有效性预言机 = 明文恢复**。

12.4 历史 TLS 防御思路

讲义引用 TLS 的防御：

1. 预先生成一个随机替代值 R ；
2. 始终执行 RSA 解密和后续处理；
3. 若填充非法，则在不泄露分支差异的情况下使用 R 作为 premaster secret；
4. 外部行为应与合法填充路径不可区分。

仅把错误文字统一为同一个提示还不够，还必须避免：

- 响应时间差；
- 连接关闭方式差；
- 后续协议行为差；
- 内存访问和异常差异。

现代新设计不应继续选择 RSAES-PKCS1-v1_5 加密。

13. RSA-OAEP

13.1 目标

OAEP 是 Optimal Asymmetric Encryption Padding。它在 RSA 运算前对消息进行随机化编码，目标包括：

- 让同一消息每次得到不同编码；
- 把消息和随机种子相互混合；
- 在解密时验证固定结构；
- 阻止对 RSA 代数结构的直接利用。

13.2 讲义中的简化结构

把消息和固定检查字段组成数据块：

$$DB = m \parallel 01 \parallel 00 \cdots 0.$$

选择随机种子 r ，使用两个掩码生成函数或随机预言机方向进行交叉混合：

$$X = DB \oplus H(r),$$

$$Y = r \oplus G(X).$$

把：

$$X \parallel Y$$

解释为 RSA 输入并计算密文。

实际标准 RSAES-OAEP 的字段和 MGF1 细节更具体，包括标签哈希、分隔字节和掩码长度，开发时应完全使用密码库提供的标准实现。

13.3 解码

私钥持有者先执行 RSA 求逆，得到 (X, Y) ：

$$r = Y \oplus G(X),$$

$$DB = X \oplus H(r).$$

然后验证 DB 中的固定格式与标签哈希。若任何检查失败，返回统一的 $\perp$ 。

13.4 安全结论

讲义给出的结论是：在把相关哈希/掩码函数建模为随机预言机，并假设 RSA 是陷门置换时，RSA-OAEP 可获得相应的 CCA 安全证明。

课程还介绍 OAEP+ 与 SAEP+，通过额外的完整性字段 $W(m, r)$ 改进或简化安全分析。理解重点是：

编码不仅要随机化消息，还必须在解密时验证编码一致性并拒绝无效密文。

13.5 OAEP 实现中的时序泄露

若实现按以下方式提前返回：

1. RSA 求逆结果超出允许范围时立即退出；
2. OAEP 固定字段错误时在另一个位置退出；
3. 标签哈希错误时又走第三条路径；

攻击者可能通过时间差判断失败类型，并把解密器变成新的预言机。

因此：

- 使用成熟密码库；
- 统一错误输出；
- 尽量使用常数时间验证；
- 在认证完成前不返回任何部分明文；
- 不自行实现 RSA-OAEP。

14. PKCS#1 v1.5 与 OAEP 对照

比较项	PKCS#1 v1.5 加密	RSA-OAEP
随机化	随机非零填充	随机种子与双向掩码混合
解密检查	检查块类型和分隔结构	检查标签哈希、分隔和整体结构
经典风险	Bleichenbacher 填充预言机	错误或时序泄露仍可破坏实现
安全证明	传统格式缺乏理想的现代结论	随机预言机模型中有 CCA 分析
新系统选择	不建议用于新加密设计	需要 RSA 加密时使用标准库 OAEP

即使使用 OAEP，也不意味着可以忽略密钥管理、随机数质量、侧信道和参数选择。

15. RSA 私钥指数不能太小

15.1 性能诱惑

RSA 解密需要计算：

$$c^d \pmod{N}.$$

较小的 d 意味着较少的平方-乘步骤，看起来可以加速解密。

15.2 Wiener 攻击

由：

$$ed \equiv 1 \pmod{\varphi(N)},$$

存在整数 k ：

$$ed - k\varphi(N) = 1.$$

当 p, q 大小接近时：

$$\varphi(N) = N - p - q + 1$$

与 N 很接近。因此：

$$\frac{e}{N} \approx \frac{k}{d}.$$

若 d 足够小， k/d 会出现在 e/N 的连分数收敛项中，攻击者可从公开 (N, e) 恢复 d 。

讲义给出的经典界包括：

- Wiener： $d < N^{0.25}$ 附近不安全；
- Boneh-Durfee：攻击范围扩展到约 $d < N^{0.292}$ 。

结论是：不能通过任意缩小私钥指数来加速 RSA。

16. 低公开指数

16.1 为什么公开指数可以较小

RSA 加密或验签计算：

$$m^e \pmod{N}.$$

选择二进制中 1 很少的公开指数可以减少乘法次数。讲义推荐：

$$e = 65537 = 2^{16} + 1.$$

其二进制结构简单，同时避免了许多极小指数的历史问题。

16.2 小 e 不会修复教科书 RSA

使用 $e = 65537$ 并不能让教科书 RSA 安全。随机化填充和安全编码仍然必不可少。

当 $e = 3$ 且消息未正确随机填充时，还可能出现：

- 小明文直接开整数立方根；
- 同一明文向多个模数加密时的广播攻击；
- 代数相关消息攻击。

因此，现代常见做法是使用 $e = 65537$ ，并配合标准 RSA-PSS 签名或 RSA-OAEP 加密，而不是裸 RSA。

17. RSA 密钥长度

讲义用以下表格比较对称安全级别与 RSA 模数大小：

对称安全级别	RSA 模数大小
80 位	1024 位
128 位	3072 位
256 位	15360 位

这说明 RSA 参数远大于对称密钥，且安全级别不会与模数长度线性对应。

其中 1024 位 RSA 和 80 位安全属于历史参数，不适合新系统。实际部署应遵循当前协议标准、合规要求和成熟密码库，而不是仅依据课程旧表自行选择。

18. RSA 实现攻击

数学上的 RSA 假设成立，不代表实现不会泄露私钥。

18.1 时序攻击

平方-乘、模约减或 CRT 运算的时间可能依赖私钥位。攻击者收集大量运行时间并进行统计，可能恢复 d 。

常见防御包括：

- RSA blinding;
- 常数时间大整数运算;
- 避免秘密相关分支与内存访问;
- 成熟密码库实现。

18.2 功耗攻击

智能卡执行平方和乘法时的功耗模式可能不同。攻击者测量功耗轨迹，推断私钥指数位。

需要算法随机化、硬件防护、掩码和侧信道审计。

18.3 故障攻击

攻击者通过电压、时钟、激光等方式使设备在私钥运算中产生错误。错误输出与正确数学关系结合，可能直接泄露因数。

一个重要防御是：在返回 RSA 私钥运算结果前，用公钥重新验证结果。

19. CRT 故障攻击

19.1 CRT 加速解密

直接计算：

$$x = c^d \pmod{N}$$

较慢。若知道 p, q ，可分别计算：

$$x_p = c^d \pmod{p},$$

$$x_q = c^d \pmod{q},$$

再通过中国剩余定理组合成模 N 的结果。由于两个模数约为 N 的一半比特长度，这通常显著加速运算。

19.2 单分支故障

假设 x_p 正确，但 x_q 出错。组合后输出 x' 满足：

$$x' \equiv c^d \pmod{p},$$

但：

$$x' \not\equiv c^d \pmod{q}.$$

使用公开指数验证：

$$(x')^e \equiv c \pmod{p},$$

但：

$$(x')^e \not\equiv c \pmod{q}.$$

因此：

$$\gcd((x')^e - c, N) = p$$

或得到另一个因数 q ，取决于哪一支正确。

攻击者只需一个故障输出就可能分解 N 。

19.3 防御

在释放输出前检查：

$$(x')^e \stackrel{?}{=} c \pmod{N}.$$

若不相等，丢弃结果。讲义指出这种检查带来一定性能开销，但比泄露完整私钥的代价小得多。

20. RSA 密钥生成中的随机数问题

20.1 共享素数

若设备启动时熵不足，多个设备可能生成相同素数 p ，但另一个素数不同：

$$N_1 = pq_1,$$

$$N_2 = pq_2.$$

攻击者只需计算：

$$\gcd(N_1, N_2) = p.$$

于是两个模数都被分解，两个私钥都可恢复。

20.2 为什么单个公钥看起来正常

仅检查 N_1 本身，很难发现其素数也被另一设备使用。问题只有在比较大量公开模数时才显现。

讲义引用的历史扫描曾分解约 0.4% 的公开 HTTPS RSA 密钥，说明密钥生成随机性不是理论细节。

20.3 工程要求

- 使用操作系统提供的密码学安全随机源；
- 确保系统熵池已正确初始化；
- 不自行编写 PRNG 和素数生成器；
- 在批量设备制造和首次启动场景中特别检查熵来源；
- 使用经过审计的密码库生成 RSA 密钥。

21. 安全层次对照

对象或方案	提供什么	缺少什么
RSA 陷门置换	有陷门时可逆，无陷门时假设难逆	随机化、语义安全、CCA 安全
教科书 RSA	正确恢复整数消息	确定且可篡改，不是安全加密

对象或方案	提供什么	缺少什么
PKCS#1 v1.5 加密	随机填充与基本格式检查	实现易形成 Bleichenbacher 预言机
RSA-OAEP	随机化编码和结构验证	仍依赖正确实现、随机数和参数
TDF + KDF + AEAD	混合加密, 随机预言机模型下可分析 CCA	依赖具体构造和假设
RSA 数学实现	模幂正确	仍可能受时序、功耗、故障攻击

22. 常见误区

误区 1: 单向函数就是安全加密

错误。单向性只保证难以恢复完整输入, 不保证隐藏输入的某个比特, 也不保证抵抗关系测试和篡改。

误区 2: RSA 是公钥加密方案

不够准确。RSA 首先是陷门置换。必须配合 OAEP、KEM 等标准构造才能成为安全加密方案。

误区 3: 私钥就是 p 和 q , d 不算私钥

错误。 d 和 CRT 参数都属于秘密材料。知道任一足够的陷门信息都可能恢复全部私钥能力。

误区 4: 教科书 RSA 只是不够随机, 除此之外没有问题

错误。它还具有乘法可篡改性、小空间枚举、相关消息等多种结构性攻击。

误区 5: PKCS#1 v1.5 有随机填充, 所以一定 CCA 安全

错误。解密端的格式响应可形成 Bleichenbacher 预言机并恢复目标明文。

误区 6: 统一显示同一个错误字符串就消除了预言机

错误。时间、连接状态、后续协议行为和资源访问仍可能泄露内部错误类型。

误区 7: OAEP 的安全意味着自己实现也安全

错误。安全证明针对抽象算法和模型。错误编码、提前返回、非恒定时间检查都会破坏实现。

误区 8: RSA 求逆困难已被证明等价于因数分解困难

错误。因数分解可用于 RSA 求逆, 但一般反向归约尚未得到证明。

误区 9: 减小私钥 d 是安全的性能优化

错误。小 d 会受到 Wiener 和 Boneh-Durfee 等攻击。

误区 10: 只要公开指数 e 小, RSA 就不安全

错误。标准的 $e = 65537$ 很常见。危险来自极小指数与错误或缺失填充、相关消息等条件的结合。

误区 11: CRT 只是无安全影响的性能优化

错误。CRT 单分支故障可能通过一个错误输出分解 N ，必须进行结果验证和故障防护。

误区 12：素数足够大，随机数质量就不重要

错误。若设备重复生成同一个素因子，攻击者用一次 gcd 就能分解多个模数。

误区 13：课程中的 1024 位 RSA 仍适合新系统

错误。它是历史教学参数。真实部署应遵循当前标准，优先使用现代协议和经过审计的实现。

23. 必记公式

公钥加密正确性：

$$D(sk, E(pk, m)) = m.$$

陷门函数正确性：

$$F^{-1}(sk, F(pk, x)) = x.$$

混合加密：

$$\begin{aligned} x &\leftarrow X, & y &= F(pk, x), \\ k &= H(x), & c &= E_s(k, m), \\ C &= (y, c). \end{aligned}$$

RSA 密钥关系：

$$\begin{aligned} N &= pq, & \varphi(N) &= (p-1)(q-1), \\ ed &\equiv 1 \pmod{\varphi(N)}. \end{aligned}$$

RSA 正向与逆向：

$$\begin{aligned} y &= x^e \pmod{N}, \\ x &= y^d \pmod{N}. \end{aligned}$$

教科书 RSA 乘法关系：

$$m_1^e m_2^e \equiv (m_1 m_2)^e \pmod{N}.$$

Bleichenbacher 修改密文：

$$\begin{aligned} c' &= r^e c \pmod{N}, \\ (c')^d &= rx \pmod{N}. \end{aligned}$$

CRT 故障分解：

$$\gcd((x')^e - c, N) = p \text{ 或 } q.$$

共享素数：

$$N_1 = pq_1, \quad N_2 = pq_2,$$

$$\gcd(N_1, N_2) = p.$$

24. 自测题

题目

1. 公钥加密为什么通常必须随机化？
2. IND-CCA 攻击者可以查询哪些密文的解密结果？
3. 为什么公钥加密不能要求攻击者无法制造新密文？
4. 陷门函数的三个算法分别做什么？
5. 陷门置换比一般陷门函数多了什么要求？
6. TDF + KDF + AEAD 构造中， y 和 c 分别承担什么任务？
7. 为什么只知道 $y = F(pk, x)$ 时， $H(x)$ 可以用作对称密钥？
8. 为什么不能直接令 $c = F(pk, m)$ ？
9. RSA 密钥生成中为什么要求 $\gcd(e, \varphi(N)) = 1$ ？
10. 证明 $(x^e)^d = x \pmod{N}$ 的关键定理是什么？
11. RSA 假设与因数分解假设是什么关系？
12. 教科书 RSA 为什么不满足 IND-CPA？
13. RSA 的乘法同态如何让攻击者构造相关明文密文？
14. Bleichenbacher 攻击需要解密服务器泄露什么信息？
15. 查询 $c' = r^e c$ 后，服务器内部解密的明文与原明文有什么关系？
16. OAEP 为什么同时需要随机种子和格式检查？
17. 为什么 OAEP 错误分支的时间差仍然危险？
18. 小私钥指数 d 为什么可能从公开 (N, e) 中恢复？
19. 为什么 $e = 65537$ 常被选作公开指数？
20. CRT 解密中一个模 q 分支故障怎样泄露 p ？
21. 两个 RSA 模数共享素因子时如何同时分解？
22. 为什么 RSA 密钥生成必须使用正确初始化的 CSPRNG？

参考答案

1. 攻击者知道公钥，可以自行加密候选消息；若加密确定，就能与挑战密文直接比较。
2. 挑战前后均可查询任意密文，但挑战后不能把挑战密文本身原样提交。
3. 任何人知道 pk 都能运行 $E(pk, m)$ ，合法生成新的密文。
4. G 生成公开描述和陷门， F 公开正向计算， F^{-1} 使用陷门求逆。
5. 固定公钥后，函数必须是定义域到值域的一一映射，通常是同一集合上的置换。
6. y 封装随机秘密 x ； c 使用从 x 派生的对称密钥认证加密实际消息。
7. 陷门函数单向性使攻击者难以恢复随机 x ；在随机预言机模型中，未查询到 x 就无法预测 $H(x)$ 。
8. 它是确定性的，可能泄露消息关系，并继承具体函数的代数可篡改结构。
9. 这样 e 才在模 $\varphi(N)$ 下有逆元 d ，指数映射才可逆。
10. Euler 定理，以及 $ed = 1 + k\varphi(N)$ 。
11. 分解 N 后可求逆 RSA；但一般 RSA 求逆是否必然能推出因数分解尚未证明。
12. 攻击者可以计算 m_0^e, m_1^e 并与挑战密文比较。
13. 把 c 乘以 r^e 得到 $(mr)^e$ ，解密后明文变成 mr 。
14. 修改后密文解密结果是否具有合法 PKCS#1 v1.5 填充或等价的可区分为行。

15. 若原明文整数为 x ，则新密文解密为 $rx \bmod N$ 。
16. 随机种子消除确定性并混合消息；格式检查让解密方拒绝被篡改或不一致的编码。
17. 攻击者可把时间差当作错误类型预言机，通过大量自适应查询恢复明文。
18. $ed - k\varphi(N) = 1$ 且 $\varphi(N)$ 接近 N ，小 d 时 k/d 可由 e/N 的连分数逼近恢复。
19. 它的二进制形式简单，加密和验签较快，同时避免了许多 $e = 3$ 的历史陷阱。
20. 错误输出 x' 在模 p 下仍正确，所以 $(x')^e - c$ 被 p 整除，但通常不被 q 整除；取与 N 的 gcd 得到 p 。
21. 计算 $\gcd(N_1, N_2)$ ，结果就是共享素因子。
22. 熵不足会让不同设备重复生成素因子，公开模数之间一次 gcd 就可能恢复私钥。

25. 一页复习提纲

1. 公钥加密由 G, E, D 组成，加密必须随机化才能达到普通语义安全。
2. IND-CCA 允许攻击者查询除挑战密文本身外的其他密文解密结果。
3. 陷门函数公开易算、无陷门难逆、有陷门易逆；陷门置换还要求双射。
4. 单向性不等于语义安全，不能直接把陷门函数应用到消息上。
5. 混合构造随机选择 x ，公开 $F(pk, x)$ ，用 $H(x)$ 作为 AEAD 密钥。
6. RSA: $N = pq$, $ed = 1 \bmod \varphi(N)$ ，正向 x^e ，逆向 y^d 。
7. RSA 假设认为随机 RSA e 次根难求；它尚未被证明与因数分解完全等价。
8. 教科书 RSA 确定、可枚举且具有乘法可篡改性，绝不是安全加密。
9. PKCS#1 v1.5 使用随机填充，但格式错误泄露会形成 Bleichenbacher 预言机。
10. Bleichenbacher 利用 $c' = r^e c$ ，让解密明文变成 rx ，再通过格式响应缩小 x 的范围。
11. OAEP 用随机种子和双向掩码混合消息，并在解密时验证固定结构。
12. OAEP 的抽象安全不会自动保证实现安全，错误与时序必须不可区分。
13. 私钥指数 d 不能过小；常用公开指数为 $e = 65537$ 。
14. RSA 参数表具有历史背景，1024 位 RSA 不适合新系统。
15. RSA 还可能受到时序、功耗和故障侧信道攻击。
16. CRT 单分支故障可通过 $\gcd((x')^e - c, N)$ 直接得到素因子。
17. 密钥生成熵不足会产生共享素因子，两个公开模数做 gcd 即可被分解。
18. 工程实践中使用标准密码库和现代协议，不实现裸 RSA、PKCS#1 解析或 OAEP。

26. 本课关系图

陷门函数 / 置换 F

- ├ 公开 pk : 正向 $F(pk, x)$ 容易
- ├ 没有 sk : 求 x 假设困难
- └ 拥有 sk : $F^{-1}(sk, y)$ 容易

不能直接加密消息
(确定性、可篡改)

随机 $x \rightarrow F(pk, x)$ 作为头部
 $\rightarrow H(x)$ 作为对称密钥
 $\rightarrow$ AEAD 加密实际消息

↓
▼
混合公钥加密

RSA 实例:

$N=pq$, $ed=1 \bmod \phi(N)$, $F(x)=x^e \bmod N$

- |
- ├─ 裸 RSA: 不安全
- ├─ PKCS#1 v1.5: 填充预言机风险
- └─ OAEP: 标准随机化编码, 但必须正确实现

↓
▼
还需防御: 小 d 、时序、功耗、CRT 故障、弱随机数

本课最重要的结论是: **密码原语的困难性只是起点。安全公钥加密来自经过证明的组合方式、随机化编码、认证失败处理和没有侧信道的实现, 而不是一次裸模幂运算。**